**CENTRO DE ENSINO SUPERIOR DE MARINGÁ – UNICESUMAR**
**CAMPUS PONTA GROSSA**
**CENTRO DE CIÊNCIAS EXATAS, TECNOLÓGICAS E AGRÁRIAS**
**ESCOLA DE TECNOLOGIA DE INFORMAÇÃO**

**BACHARELADO EM ENGENHARIA DE SOFTWARE**
**TECNOLOGIA EM ANÁLISE E DESENVOLVIMENTO DE SISTEMAS**

**ESTRUTURA DE DADOS I – NOTAS DE AULA 1**
**Prof. MSc. Gabriel Passos de Jesus (2026/1)**

# REVISÃO DE LINGUAGEM "C" E CONCEITOS INTRODUTÓRIOS

---

## TÓPICO 1 – POR QUE REVISAR LINGUAGEM "C" EM UMA DISCIPLINA DE ESTRUTURA DE DADOS?

Estruturas de Dados (EDs) envolvem:

- Organização e armazenamento eficiente de dados
- Manipulação direta da memória
- Implementação de estruturas dinâmicas (listas, pilhas, filas, árvores)

A linguagem **C** é ideal porque:

- Permite controle explícito da memória
- Trabalha diretamente com ponteiros
- Não possui estruturas prontas como em linguagens de alto nível (ex: Java, Python), o que força a compreensão interna das EDs
    - NOTA: Quem fez a disciplina de Programação e utilizou a linguagem "C" deve se lembrar do debate sobre a diferença entre alto e baixo nível.
    - NOTA: Sim, é possível trabalhar com EDs em outras linguagens (isso é visto no livro do Thiago Leite e Carvalho, publicado em 2022, no entanto, a linguagem "C" é vista como o "esqueleto" de tudo o que se sabe sobre EDs).

**QUESTÃO:** Qual a diferença entre alto e baixo nível? Por que a linguagem baixo nível é chamada, por alguns autores, de "linguagem de máquina"?.

## TÓPICO 2 – TIPOS DE DADOS EM “C”

### TIPOS PRIMITIVOS

| Tipo | Descrição | Tamanho típico* |
|---|---|---|
| int | Números inteiros | 4 bytes |
| float | Ponto flutuante simples | 4 bytes |
| double | Ponto flutuante dupla precisão | 8 bytes |
| char | Caractere | 1 byte |
| void | Ausência de tipo | — |

Se observarmos a tabela acima, notamos que há uma observação no chamado “Tamanho Típico”. Vamos entender o porquê.

Em primeiro lugar, Em C, o padrão da linguagem (definido pela **norma ISO C**) **não fixa o tamanho exato em bytes da maioria dos tipos primitivos**.

Ele define apenas:

- Relações mínimas entre tamanhos
- Intervalos mínimos de representação
- Hierarquia entre tipos

Desta forma, a norma ISO C garante somente o que é previsto na linha de código abaixo:

```
sizeof(char) == 1
```

```
sizeof(short) ≤ sizeof(int) ≤ sizeof(long) ≤ sizeof(long long)
```

Ela define apenas que:

- int deve representar pelo menos -32767 a +32767
- long deve representar pelo menos -2147483647 a +2147483647

Ou seja: o padrão define **capacidade mínima**, não tamanho fixo.

**QUESTÃO:** O que é cláusula “sizeof”?

Assim, provamos que depende da arquitetura do *hardware*.

O tamanho dos tipos depende principalmente de:

- Arquitetura do processador (16, 32 ou 64 bits)
- Modelo de compilador
- Sistema operacional
- Convenções ABI (Application Binary Interface)

Vamos trazer alguns exemplos históricos:

EX 1 – 16 bits

int → 2 bytes

long → 4 bytes

EX 2 – 32 bits

int → 4 bytes

long → 4 bytes

EX 3 – 64 bits (LP64 – usual em algumas distribuições Linux ou no macOS)

int → 4 bytes

long → 8 bytes

long long → 8 bytes

EX 4 – 64 bits (LLP64 – usual em sistemas Windows)

int → 4 bytes

long → 4 bytes

long long → 8 bytes

QUESTÃO: Qual a razão para esta variação?

**DEMONSTRAÇÃO DE CÓDIGO – EM "C"**

```
#include <stdio.h>

int main() {
    printf("char: %lu bytes\n", sizeof(char));
    printf("short: %lu bytes\n", sizeof(short));
    printf("int: %lu bytes\n", sizeof(int));
    printf("long: %lu bytes\n", sizeof(long));
    printf("long long: %lu bytes\n", sizeof(long long));
    printf("float: %lu bytes\n", sizeof(float));
    printf("double: %lu bytes\n", sizeof(double));

    return 0;
}
```

Porém, percebemos, neste código, que ocorre o chamado PROBLEMA DA PORTABILIDADE, que pode ser resolvido através das seguintes manipulações:

Iniciamos o código com a seguinte forma:

```
#include <stdint.h>
```

Definimos os tipos:

```
int8_t
int16_t
int32_t
int64_t
uint32_t
```

Executamos o seguinte código:

```
#include <stdio.h>
#include <stdint.h>

int main() {
    int32_t x = 100;
    printf("int32_t: %lu bytes\n", sizeof(x));
}
```

## TÓPICO 3 - DEFININDO CONCEITOS – TIPOS PRIMITIVOS VS TIPOS ABSTRATOS

### TÓPICO 3.1 – DADOS PRIMITIVOS

**Dados Primitivos**

- Representam valores simples
- São indivisíveis no nível da linguagem
- Operações básicas já definidas pelo compilador

Exemplos: int, float, char

### TÓPICO 3.1. – DADOS ABSTRATOS

Um **Tipo Abstrato de Dados (TAD)** define:

- Um conjunto de valores
- Um conjunto de operações sobre esses valores
- Sem expor detalhes de implementação

Exemplos clássicos:

- Pilha (Stack)
- Fila (Queue)
- Lista
- Árvore

Agora, vamos implementar na prática usando duas cláusulas distintas: struct e typedef

EX 1 – USANDO “STRUCT”

```
#include <stdio.h>

struct Aluno {
    int matricula;
    char nome[50];
    float nota;
};

int main() {
    struct Aluno a1;

    a1.matricula = 123;
    a1.nota = 8.5;

    printf("Matrícula: %d\n", a1.matricula);
    printf("Nota: %.2f\n", a1.nota);

    return 0;
}
```

EX 2 – USANDO “TYPEDEF”

```
typedef struct {
    int matricula;
    char nome[50];
    float nota;
} Aluno;

int main() {
    Aluno a1;
    a1.matricula = 456;
}
```

**QUESTÃO:** Qual a diferença entre “struct” e “typedef”?

## TÓPICO 4 – MEMÓRIA E VARIÁVEIS

Aqui, vamos discutir este tópico a partir de uma única questão a partir da linha abaixo:

```
int x = 10;
```

**QUESTÃO:** O que ocorre quando fazemos a declaração expressa acima?

## TÓPICO 5 – PONTEIROS

Por definição, ponteiro é uma variável que armazena o **endereço de memória** de outra variável.

Declaramos o ponteiro da seguinte forma:

```
int *p;
```

No entanto, em se tratando de “C” temos dois operadores para ponteiro:

| **Operador** | **Função** |
|---|---|
| & | Obtém o endereço |
| * | Acessa o valor apontado |

**QUESTÃO:** Qual a diferença entre os operadores?

PS: Não vale dizer que é o que está escrito em “Função” na tabela acima :D

Vamos demonstrar ponteiros na prática, a partir do código abaixo:

```
#include <stdio.h>

int main() {
    int x = 10;
    int *p;

    p = &x;

    printf("Valor de x: %d\n", x);
    printf("Endereço de x: %p\n", &x);
    printf("Valor de p: %p\n", p);
    printf("Valor apontado por p: %d\n", *p);

    return 0;
}
```

Agora, vamos ao código abaixo:

```
#include <stdio.h>

void altera(int *n) {
    *n = 100;
}

int main() {
    int x = 10;
    altera(&x);
    printf("x = %d\n", x);
    return 0;
}
```

**QUESTÃO:** No código acima, qual o tipo de passagem que ocorre?

## TÓPICO 6 – ALOCAÇÃO DINÂMICA DA MEMÓRIA

Trabalharemos, essencialmente, com a biblioteca <stdlib.h>

A alocação dinâmica permite reservar memória **em tempo de execução**, na **Heap**, ao contrário das variáveis locais (Stack), que são alocadas em tempo de compilação.

São as principais funções:

`malloc()`

`calloc()`

`realloc()`

`free()`

## TÓPICO 6.1. – MALLOC

Dado o código:

```
void *malloc(size_t size);
```

O que a função faz?

- Reserva um bloco contínuo de memória com o número de bytes especificado.
- Retorna um ponteiro genérico (void *) para o início do bloco.
- Não inicializa a memória.
- Retorna NULL se não conseguir alocar.

**QUESTÃO:** O que ocorre se:

```
malloc(4);
```

**DEMONSTRAÇÃO VIA CÓDIGO**

```
#include <stdio.h>
#include <stdlib.h>

int main() {
    int *p;
    p = (int *) malloc(sizeof(int));
    if (p == NULL) {
        printf("Erro de alocação\n");
        return 1;
    }

    *p = 42;
    printf("Valor: %d\n", *p);
    free(p);
    return 0;
}
```

Boas práticas:

- Sempre verificar se retornou `NULL`
- Sempre usar `sizeof(tipo)`
- Não esquecer de usar `free`

**TÓPICO 6.2 – CALLOC**

Dado o código:

```
void *calloc(size_t num, size_t size);
```

O que o comando acima faz:

- Aloca memória para um vetor de elementos
- Inicializa todos os bytes com zero
- Retorna ponteiro para o início do bloco

- Retorna NULL em caso de falha

Diferenças entre MALLOC e CALLOC

| **Malloc** | **calloc** |
|---|---|
| Recebe número total de bytes | Recebe quantidade de elementos e tamanho |
| Não inicializa | Inicializa com zero |

Dado o código abaixo, responda à questão abaixo:

```
#include <stdio.h>
#include <stdlib.h>

int main() {
    int *v;
    int i;

    v = (int *) calloc(5, sizeof(int));

    if (v == NULL) {
        printf("Erro de alocação\n");
        return 1;
    }

    for(i = 0; i < 5; i++) {
        printf("%d ", v[i]);
    }

    free(v);
    return 0;
}
```

**QUESTÃO:** Sem executar o código no compilador, **RESPONDA,** qual a saída?

**TÓPICO 6.3. – REALLOC**

Dado o código:

```
void *realloc(void *ptr, size_t new_size);
```

O que o comando acima faz:

- Redimensiona um bloco previamente alocado
- Pode:
    - Expandir
    - Reduzir
- Pode mover o bloco para outro endereço
- Preserva o conteúdo anterior (até o menor tamanho entre antigo e novo)

Este funcionamento pode ser explicado pelo funcionamento interno. Para isso vamos discutir a ideia de ESPAÇOS ADJACENTES.

**DEMONSTRAÇÃO DE CÓDIGO:**

```
#include <stdio.h>
#include <stdlib.h>

int main() {
    int *v;
    int i;

    v = (int *) malloc(3 * sizeof(int));

    for(i = 0; i < 3; i++) {
        v[i] = i + 1;
    }

    v = (int *) realloc(v, 6 * sizeof(int));

    for(i = 3; i < 6; i++) {
```

```
        v[i] = i + 1;
    }

    for(i = 0; i < 6; i++) {
        printf("%d ", v[i]);
    }

    free(v);

    return 0;
}
```

**AVISO:** Este código acima, fere uma boa prática de programação.

**QUESTÃO: EXPLIQUE** o porque a prática abaixo não é considerada boa prática:

```
v = realloc(v, novo_tamanho);
```

**TÓPICO 6.4. – FREE**

Dado o código abaixo:

```
void free(void *ptr);
```

Este código:

- Libera um bloco previamente alocado
- Retorna o espaço à Heap
- Não altera automaticamente o ponteiro

No entanto, temos três erros comuns, veja a tabela:

| **ERRO** | **EXEMPLO** |
|---|---|
| Memory Leak | `int *p = malloc(sizeof(int));` |
| Double Free | `free(p);`<br>`free(p);` |
| Dangling Pointer | `free(p);`<br>`*p = 10;` |

Nos três erros, ocorre o chamado COMPORTAMENTO INDEFINIDO.